Q39. LATIN SQUARE COMPLETION

(Constraint Satisfaction / NP-Complete Problem)

Register Number: 192424316

Student Name: VANGIMALLA BALA SIVA PRASAD REDDY

Aim

To solve the Latin Square Completion problem by modeling it as a Constraint Satisfaction Problem (CSP), clearly define the variables, domains, and uniqueness constraints, analyze how constraints reduce the search space, develop a backtracking-based solution, apply constraint propagation and search heuristics, justify the correctness of the solution, and evaluate its computational complexity and practical applications.

Problem Statement

A Latin square of order n is an n × n arrangement of n different symbols in which every symbol occurs exactly once in every row and exactly once in every column. The Latin Square Completion problem starts with a partially filled square and requires all empty cells to be filled while preserving the values that are already present.

The solution must satisfy two fundamental uniqueness conditions: no symbol may be repeated in a row and no symbol may be repeated in a column. Because a large number of possible assignments may exist, an intelligent search method is required. A Constraint Satisfaction Problem formulation combined with constraint propagation and backtracking provides a systematic solution.

Introduction

Latin squares are important combinatorial structures with applications in mathematics, scheduling, experimental design, cryptography, and computer science. A Latin square can be viewed as a matrix where the entries are selected from a fixed set of symbols and arranged according to strict uniqueness rules.

When some entries are missing, completing the square becomes a search problem. A brute-force algorithm may attempt every possible value in every empty cell, causing an enormous number of combinations. The CSP approach improves the process by maintaining a domain of possible values for each empty cell and continuously removing values that violate the constraints.

Backtracking is then used to explore only the remaining legal choices. Techniques such as forward checking and the Minimum Remaining Values (MRV) heuristic make the search more efficient by identifying contradictions early and selecting the most constrained cells first.

Objectives

- Represent each unfilled cell as a CSP variable.
- Define the domain of possible symbols for every variable.
- Preserve all values supplied in the original partially filled square.
- Ensure that every row contains each symbol exactly once.
- Ensure that every column contains each symbol exactly once.
- Use constraint propagation to remove impossible candidates.
- Develop a systematic backtracking search procedure.
- Use MRV and forward checking to improve search efficiency.
- Prove that the algorithm returns a valid completion when one exists.
- Analyze the time and space requirements of the approach.
- Identify practical applications, advantages, and limitations.

Latin Square Definition

For a Latin square of order n, the set of allowed symbols is normally {1, 2, ..., n}. The final square must contain every symbol exactly once in every row and every column.

Example of a valid Latin square of order 4:

1 2 3 4

3 4 1 2

2 3 4 1

4 1 2 3

In this example, every row contains 1, 2, 3, and 4 exactly once. The same is true for every column.

Constraint Satisfaction Problem Formulation

The Latin Square Completion problem can be represented using the three main components of a CSP: variables, domains, and constraints.

Variables

Every empty cell in the partially filled square is represented by a variable X[i][j], where i identifies the row and j identifies the column. Filled cells are treated as fixed variables whose values cannot be changed.

X[i][j] = value assigned to row i, column j

Domains

The domain of an empty variable is the set of symbols that can potentially be assigned to that cell. Initially, for an order-n square, the domain can be {1, 2, ..., n}. As constraints are applied, the domain becomes smaller.

Initial Domain for order 4 = {1, 2, 3, 4}

Constraints

The constraints define which assignments are legal. A candidate value is valid only if it satisfies all relevant constraints.

Row Uniqueness Constraint

Two different cells in the same row cannot contain the same symbol.

$X[i][j] \neq X[i][k]$ when $j \neq k$

Column Uniqueness Constraint

Two different cells in the same column cannot contain the same symbol.

$X[i][j] \neq X[k][j]$ when $i \neq k$

Domain Constraint

Every cell must contain one of the allowed symbols.

$$1 \leq X[i][j] \leq n$$

Fixed-Cell Constraint

Any value that was already present in the input must remain unchanged during the solution process.

Given Partially Filled Latin Square

Consider the following order-4 partially filled Latin square:

1 2 3 _

3 4 _ 2

_ 3 4 1

4 _ 2 3

The available symbols are {1, 2, 3, 4}. The empty cells have to be completed without repeating any symbol in their corresponding row or column.

Initial Candidate Domains

Before making assignments, each empty cell can initially contain any of the four symbols. Therefore, the initial domain of each empty cell is:

{1, 2, 3, 4}

The row and column constraints are then used to remove values that are already present.

Constraint Propagation

Constraint propagation is the process of reducing the domains of variables by using known constraints. It prevents the algorithm from trying values that are already impossible.

Candidates = Allowed Symbols − Row Symbols − Column Symbols

Cell at Row 1, Column 4

Row 1 contains: {1, 2, 3}

Missing value in row 1: {4}

Therefore X[1][4] = 4

The first row is now complete.

Cell at Row 2, Column 3

Row 2 contains: {3, 4, 2}

Missing value in row 2: {1}

Therefore X[2][3] = 1

Cell at Row 3, Column 1

Row 3 contains: {3, 4, 1}

Missing value in row 3: {2}

Therefore X[3][1] = 2

Cell at Row 4, Column 2

Row 4 contains: {4, 2, 3}

Missing value in row 4: {1}

Therefore X[4][2] = 1

Completed Latin Square

1 2 3 4

3 4 1 2

2 3 4 1

4 1 2 3

The square is now complete. Every row and every column contains all four symbols exactly once.

Detailed Search Space Analysis

The major challenge in Latin Square Completion is the potentially huge search space. If there are k empty cells and every cell has n possible values before constraints are considered, a simple brute-force approach can examine up to n^k assignments.

For example: n = 4 and k = 8

Possible assignments = 4^8 = 65,536

This number grows rapidly as the order of the square increases. However, row and column constraints eliminate many assignments before they are fully explored.

For example, a cell may initially have {1,2,3,4}. If 1 and 2 are already present in its row, its domain becomes {3,4}. If 3 is already present in its column, the domain becomes {4}. The algorithm can immediately assign 4 without exploring three invalid alternatives.

Backtracking-Based Solution

Constraint propagation may not always determine every cell. When multiple candidates remain, backtracking is used. Backtracking constructs a solution incrementally and reverses an assignment whenever that assignment causes a contradiction.

1. Locate an unassigned cell.
2. Generate all currently valid candidates for that cell.
3. Select a candidate value.
4. Assign the value temporarily.
5. Check the row and column constraints.
6. Propagate the consequences of the assignment.
7. If no domain becomes empty, recursively continue.
8. If a contradiction occurs, undo the assignment.
9. Try the next candidate.

10. Continue until all cells are filled or all possibilities have been exhausted.

Backtracking Pseudocode

```
BACKTRACK(square):

    if there are no empty cells:

        return TRUE


    select an empty cell

    candidates = possible values for the cell


    for each value in candidates:

        if value satisfies row and column constraints:

            place value

            propagate constraints


            if BACKTRACK(square) = TRUE:

                return TRUE


            remove value


    return FALSE
```

Forward Checking

Forward checking improves backtracking by examining the future consequences of every assignment. When a value is placed in a cell, the same value is removed from the domains of all other empty cells in that row and column.

If any neighboring cell loses every possible candidate, the current assignment is immediately identified as a contradiction. The algorithm then backtracks instead of continuing deeper into an impossible branch.

> Assignment → Remove candidate from neighbors → Check for empty domain → Backtrack if contradiction

Minimum Remaining Values Heuristic

The Minimum Remaining Values heuristic, commonly called MRV, chooses the empty cell with the smallest number of legal candidates. This is useful because the most constrained variable is most likely to expose a contradiction early.

> Cell A → {1,2,3}
>
> Cell B → {2}
>
> Cell C → {1,3}
>
> Cell D → {1,2,4}
>
> MRV selects Cell B because it has only one candidate.

Degree Heuristic

If several cells have the same minimum number of candidates, a degree heuristic can be used. It selects the cell that is involved in the largest number of remaining constraints. Assigning such a cell affects more neighboring variables and can reduce the search tree more quickly.

Constraint Propagation Strategy

A strong solver repeatedly performs the following cycle:

1. Calculate candidate domains

2. Remove row conflicts
3. Remove column conflicts
4. Assign single-candidate cells
5. Propagate the new assignment
6. Detect contradictions
7. Use MRV when branching is necessary

This approach combines deterministic deductions with systematic search.

Correctness Analysis

Validity of Every Assignment

The algorithm assigns a value to a cell only when that value is permitted by its current row and column constraints. Therefore, every assignment made by the search is locally valid.

Preservation of Given Values

The initially filled cells are never modified. Thus, the completed square always respects the original input.

Base Case

When no empty cells remain, the algorithm has reached a complete assignment. Since every assignment was checked against row and column constraints, all rows and columns satisfy the uniqueness requirements. Therefore, the result is a valid Latin square.

Backtracking Case

If a candidate eventually produces a contradiction, the algorithm removes that candidate and returns to the previous decision point. It then tries another candidate. This ensures that an incorrect partial assignment does not permanently affect the solution.

Completeness

The algorithm systematically considers every candidate that is consistent with the current constraints. Therefore, if a valid completion exists, the search can reach it. If every possible consistent assignment is exhausted, the algorithm correctly concludes that no completion exists.

Computational Complexity

A Latin square of order n contains n^2 cells. If k cells are empty, a simple brute-force upper bound is n^k because each empty cell may initially have up to n possible symbols.

Time Complexity = O(n^k)

In the worst case, k may be close to n^2, giving:

Worst-Case Time Complexity = O(n^(n^2))

This is exponential in the size of the square. Constraint propagation, forward checking, MRV, and degree-based variable selection do not remove the worst-case exponential nature, but they can substantially reduce the practical number of branches explored.

Space Complexity

The Latin square itself requires $O(n^2)$ space. If candidate domains are explicitly stored for all cells, the domain representation can require up to $O(n^3)$ space because there are n^2 cells and each may contain up to n candidates.

The recursion depth of a straightforward backtracking solver can be $O(n^2)$, because at most one assignment is made for each empty cell.

Comparison of Search Approaches

Approach	Main Idea	Effect on Search
Brute Force	Try every possible value combination	Very large search space
Basic Backtracking	Reject assignments that violate constraints	Removes invalid branches
Forward Checking	Update neighboring domains	Detects contradictions earlier

	after assignment	
MRV Backtracking	Choose the most constrained cell first	Reduces branching
Propagation + MRV	Repeated domain reduction plus intelligent variable selection	More efficient practical search

Applications

- Experimental design, where combinations of treatments must be organized without unwanted repetition.
- Scheduling and timetabling, where rows, columns, and symbols can represent people, time slots, and resources.
- Tournament scheduling and round-robin arrangements.
- Resource allocation problems with uniqueness requirements.
- Cryptographic and combinatorial constructions.
- Puzzle generation and solving.
- Pattern construction and matrix design.
- Constraint-based planning and decision systems.

Advantages

- The problem is modeled clearly using variables, domains, and constraints.
- Pre-filled values are preserved.
- Row and column uniqueness is guaranteed.
- Constraint propagation removes impossible candidates.
- Forward checking identifies contradictions early.
- MRV reduces unnecessary branching.
- Backtracking provides a complete systematic search.
- The method can determine when a valid completion does not exist.

Limitations

- The worst-case running time is exponential.

- Large-order squares can produce very large search trees.
- Poor variable selection can cause excessive backtracking.
- Maintaining candidate domains adds memory and computation overhead.
- Some highly constrained instances may still require extensive exploration.

Result

The given partially filled square:

1 2 3 _

3 4 _ 2

_ 3 4 1

4 _ 2 3

is successfully completed as:

1 2 3 4

3 4 1 2

2 3 4 1

4 1 2 3

Row Verification

Row 1 → 1 2 3 4 ✓

Row 2 → 3 4 1 2 ✓

Row 3 → 2 3 4 1 ✓

Row 4 → 4 1 2 3 ✓

Column Verification

Column 1 → 1 3 2 4 ✓

Column 2 → 2 4 3 1 ✓

Column 3 → 3 1 4 2 ✓

Column 4 → 4 2 1 3 ✓

Overall Algorithm Flow

START

↓

Read partially filled Latin square

↓

Create variables and candidate domains

↓

Apply row and column uniqueness constraints

↓

Perform constraint propagation

↓

Any single-candidate cells?

├── YES → Assign and propagate again

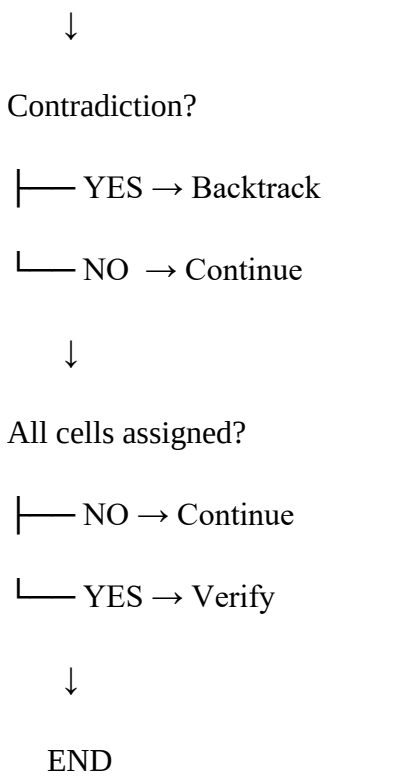
└── NO → Select cell using MRV

↓

Try candidate

↓

Forward checking



Interpretation and Insights

The main insight from this problem is that constraints are not merely rules for checking the final answer; they are also tools for controlling the search process. Each known value restricts the possible values of other cells in its row and column.

Constraint propagation can therefore transform a large combinatorial search into a sequence of smaller decisions. MRV further improves the process by selecting cells where the number of choices is already small. Forward checking prevents the algorithm from spending time on branches that have already become impossible.

This demonstrates an important principle of constraint-based optimization: stronger information about the problem structure can reduce the effective search space even when the theoretical worst-case complexity remains exponential.

Conclusion

Latin Square Completion is a classical Constraint Satisfaction Problem that requires every row and column to contain each allowed symbol exactly once. The problem can be represented using

variables for empty cells, domains containing possible symbols, and uniqueness constraints for rows and columns.

A simple brute-force method can require an exponential number of assignments. The proposed approach improves practical performance by combining constraint propagation, forward checking, MRV, degree-based variable selection, and systematic backtracking. These techniques remove invalid candidates as early as possible and reduce unnecessary exploration.

The worked example was successfully completed as a valid Latin square, and verification confirmed that every row and column contains the symbols exactly once. Although the worst-case time complexity remains exponential, the CSP-based backtracking method provides a correct, structured, and effective approach for solving partially filled Latin squares and demonstrates the value of constraint-driven search in combinatorial problems.